

To get there, we'll work iteratively. We'll begin with a simple use case that we'd like our library to handle, and then develop an interface that facilitates this use case. Only then will we consider what our implementation of this interface should be. As we keep refining our design, we'll oscillate between the interface and implementation as we gain a better understanding of the domain and the design space through progressively more complex use cases. We'll emphasize *algebraic reasoning* and introduce the idea that an API can be described by *an algebra* that obeys specific *laws*.

Why design our own library? Why not just use the concurrency primitives that come with Scala's standard library in the `scala.concurrent` package? This is partially for pedagogical purposes—we want to show you how easy it is to design your own practical libraries. But there's another reason: we want to encourage the view that no existing library is authoritative or beyond reexamination, even if designed by experts and labeled “standard.” There's a certain safety in doing what everybody else does, but what's conventional isn't necessarily the most practical. Most libraries contain a lot of arbitrary design choices, many made unintentionally. When you start from scratch, you get to revisit all the fundamental assumptions that went into designing the library, take a different path, and discover things about the problem space that others may not have even considered. As a result, you might arrive at your own design that suits your purposes better. In this particular case, our fundamental assumption will be that our library admits *absolutely no side effects*.

We'll write a lot of code in this chapter, largely posed as exercises for you, the reader. As always, you can find the answers in the downloadable content that goes along with the book.

7.1 *Choosing data types and functions*

When you begin designing a functional library, you usually have some general ideas about what you want to be able to do, and the difficulty in the design process is in refining these ideas and finding a data type that enables the functionality you want. In our case, we'd like to be able to “create parallel computations,” but what does that mean exactly? Let's try to refine this into something we can implement by examining a simple, parallelizable computation—summing a list of integers. The usual left fold for this would be as follows:

```
def sum(ints: Seq[Int]): Int =  
  ints.foldLeft(0)((a,b) => a + b)
```

Here `Seq` is a superclass of lists and other sequences in the standard library. Importantly, it has a `foldLeft` method.

Instead of folding sequentially, we could use a divide-and-conquer algorithm; see the following listing.